

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Отчёт по лабораторной работе № 1
по дисциплине
«Методология и практики DevOps»
Тема 3 «Библиотека»

Выполнили:
студенты группы 241-351
Махров И.М.
Крамской И.М.

Преподаватель: _____

Москва — 2026

Результат лабораторной работы

Разработано веб-приложение «Библиотека» версии 0.1.0 для учёта университетского книжного фонда. Оно объединяет каталог, справочники, аутентификацию, выдачи и возвраты, отчёты и журнал изменений. Приложение использует HTTP API и реляционную базу SQLite.

Цель работы

Сформулировать требования к сквозному учебному проекту и реализовать его первую работоспособную версию с воспроизводимым запуском, проверками и едиными правилами внесения изменений.

Пункт задания	Результат в комплекте
1–2	Выбрана тема 3 «Библиотека»; описаны назначение, роли, сценарии и правила.
3	Техническое задание из восьми разделов по структуре выданной методички.
4–5	Веб-интерфейс, HTTP API, SQLite; пять редактируемых связанных сущностей, пользователи и выдачи.
6	Ошибки JSON, переменные окружения, GET /health.
7	README, .gitignore, .env.example, описание API и схема данных.
8–10	Задача №1, PR №2, четыре коммита и реальный конфликт подтверждены. Тег v0.1.0 ещё требуется создать и отправить.
11	Правила изменений, обязательные проверки, запреты и порядок приёмки в CONTRIBUTING.md.
Дополнение	Регистрация, вход, выход и разграничение ролей реализованы.

Согласование темы и итоговая приёмка преподавателем выполняются отдельно. Настоящий отчёт фиксирует реализацию и результаты проверки, но не заменяет это согласование.

Навигация по отчёту

Описание реализации и интерфейса предшествует техническому заданию. В начале ТЗ размещено кликабельное содержание разделов и подпунктов. После ТЗ приведены протокол испытаний и инструкция по запуску.

Архитектура и данные

Браузер загружает HTML, CSS и JavaScript с того же сервера, к которому отправляет JSON-запросы. Flask проверяет сессию и роль, валидирует данные, выполняет SQL и возвращает результат. SQLite сохраняет состояние между запусками.

Уровень	Реализация
Интерфейс	static/index.html, static/style.css, static/app.js
Прикладная логика	app.py — маршруты, роли, правила, транзакции
Хранение	schema.sql и файл data/library.sqlite3
Эксплуатация	setup_local.py, start.ps1, /health, backup
Качество	tests/test_app.py, CONTRIBUTING.md

Связи предметной области

Автор → книги → экземпляры → выдачи. Филиал → экземпляры. Факультет → читатели → выдачи. Для всех этих связей используется отношение «один ко многим». У книги в версии 0.1.0 один основной автор. У библиотекаря факультет может отсутствовать.

Дополнительные таблицы sessions и audit_events хранят сессии и события изменения данных. Отчёты вычисляются по фонду и истории выдач. Их результаты не дублируются в отдельных таблицах.

Содержательное правило

Один физический экземпляр может иметь не более одной незавершённой выдачи. Перед выдачей выполняется BEGIN IMMEDIATE; проверки доступности, лимита и просрочек, создание выдачи и запись аудита входят в одну транзакцию. Частичный уникальный индекс по сору_id при returned_at IS NULL защищает это правило и на уровне базы.

Читатель получает не более пяти книг одновременно и не может получить новую книгу, пока имеет просроченную. Стандартный срок — 14 дней; лимит и срок задаются в окружении. Возврат сохраняет историю и освобождает экземпляр.

Защита данных

Пароли хранятся как bcrypt-хеши. В cookie передаётся случайный токен, его хеш и срок сессии хранятся на сервере. Изменяющие запросы авторизованного пользователя требуют CSRF-токен. Роль проверяется на сервере. Внешние ключи запрещают удалять используемые записи.

Веб интерфейс

Интерфейс содержит экран входа и регистрации, каталог, выдачи и возвраты, управление фондом, отчёты и журнал действий. Библиотекарю доступны служебные разделы; читателю — каталог и собственные выдачи.

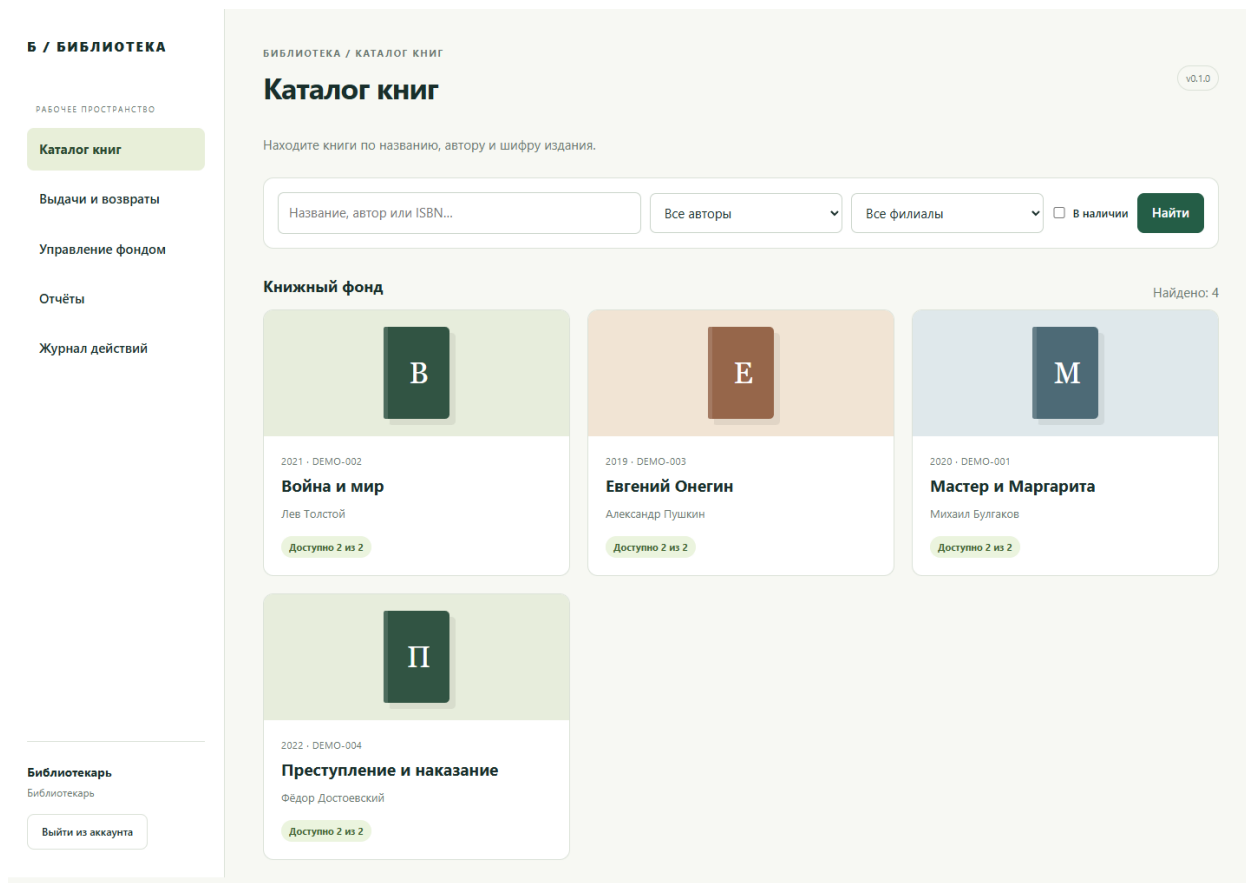


Рисунок 1 — каталог библиотекаря с поиском, фильтрами и доступностью экземпляров.

Записи справочников добавляются и изменяются через формы. Перед удалением запрашивается подтверждение. Ошибки API показываются в интерфейсе понятным сообщением; регистрация не позволяет выбрать роль библиотекаря.

Браузерная проверка выполнена на экранах 1440×1000 и 390×844. Проверены основные формы, переходы и отсутствие горизонтального выхода страницы за границы мобильного экрана.

Содержание технического задания

1 Общие положения	6
1.1 Наименование системы	6
1.2 Основание для разработки	6
1.3 Заказчик и исполнители	6
1.4 Сроки и финансирование	6
1.5 Передача результатов и внесение изменений	6
2 Назначение и цели создания продукта	7
2.1 Назначение системы	7
2.2 Цели создания системы	7
3 Характеристика объекта автоматизации	7
3.1 Описание объекта	7
3.2 Участники процесса	7
4 Требования к продукту	8
4.1 Функциональные требования	8
4.2 Правила данных	8
4.3 Требования к системе в целом	9
4.4 Надёжность и целостность	9
4.5 Безопасность и обработка ошибок	9
4.6 Виды обеспечения и программная среда	9
4.7 Требования к Git и автоматическим проверкам	10
4.8 Контейнерная среда	10
5 Состав и содержание работ	11
5.1 Порядок планирования	11
5.2 Этапы и результаты работ	11
6 Порядок контроля и приёмки системы	12
6.1 Организация испытаний	12
6.2 Критерии приёмки	12
6.3 Устранение замечаний и передача результата	12
6.4 Приёмка Git процесса	12
7 Требования к документированию	13
7.1 Форма и порядок передачи документов	13
7.2 Состав документации	13
8 Источники разработки	14
8.1 Учебные и методические материалы	14
8.2 Техническая документация	14

Техническое задание

1 Общие положения

1.1 Наименование системы. Полное наименование: «Информационная система учёта библиотечного фонда и выдачи книг». Краткое наименование: «Библиотека». Условное обозначение: LIB-LR1. Версия результата: 0.1.0.

1.2 Основание для разработки. Основание: задание к лабораторной работе №1 «Сквозной проект и Git-процесс», вариант 3 «Библиотека», включая дополнение об обязательной аутентификации от 18.09.2026. Договор и отдельный шифр темы не назначены.

1.3 Заказчик и исполнители. Учебная организация: Московский политех. Исполнители: Махров И.М., Крамской И.М., группа 241-351. Представитель заказчика и лицо, принимающее работу: преподаватель дисциплины «Методология и практики DevOps»; ФИО заполняется при сдаче.

1.4 Сроки и финансирование. Плановая длительность разработки — десять учебных дней от согласования ТЗ. Дата начала и дата сдачи устанавливаются преподавателем. Этапы и контрольные точки приведены в разделе 5. Финансирование не предусмотрено: работа выполняется в учебных целях.

1.5 Передача результатов и внесение изменений. Результат передаётся архивом с исходным кодом и документацией. Дополнения оформляются новой редакцией ТЗ с описанием изменённых требований и согласуются до приёмки. Исходники ведутся в <https://github.com/Draponoid/library-app>; основная ветка — master. Изменения проходят через задачи, отдельные ветки и pull requests. Первая принятая версия должна иметь опубликованный тег v0.1.0.

2 Назначение и цели создания продукта

2.1 Назначение системы. Система предназначена для локальной автоматизации работы учебной библиотеки: ведения фонда, поиска изданий, учёта выдач и возвратов.

2.2 Цели создания системы. Цели: устранить двойную выдачу экземпляра, обеспечить прослеживаемую историю и дать читателю доступ к собственным выдачам. Достижение целей проверяется сценариями раздела 6.

3 Характеристика объекта автоматизации

3.1 Описание объекта. Объект — библиотека университета с несколькими филиалами и читателями разных факультетов.

3.2 Участники процесса. Библиотекарь ведёт справочники и обслуживает выдачи; читатель регистрируется, ищет книги и контролирует сроки возврата; администратор установки запускает приложение, задаёт окружение и сохраняет резервные копии. Администратор установки не является отдельной ролью веб-приложения.

4 Требования к продукту

4.1 Функциональные требования

Код	Требование
F01	Регистрация создаёт только читателя с уникальным логином, именем, паролем и выбранным факультетом.
F02	Вход проверяет пароль, создаёт ограниченную по времени сессию; выход отзывает её. Неавторизованный доступ к защищённому API возвращает 401.
F03	Библиотекарь создаёт, просматривает, изменяет и удаляет авторов, книги, филиалы, экземпляры и факультеты. Используемые записи защищены от удаления.
F04	Каталог ищет по названию, автору и ISBN/шифру без учёта регистра. Фильтры автора, филиала и наличия комбинируются.
F05	Выдача связывает читателя со свободным экземпляром. Срок вычисляет сервер. Одна активная выдача на экземпляр; по умолчанию лимит 5 книг и срок 14 дней. Просрочка блокирует новую выдачу.
F06	Возврат закрывает активную выдачу и сохраняет историю. Повторный возврат и изменение выданного экземпляра запрещены.
F07	Библиотекарь видит все выдачи; читатель — только свои. Служебные операции защищены проверкой роли и возвращают 403 при нарушении. Список выдач поддерживает status=all, active, returned, overdue; недопустимый статус возвращает 422.
F08	Отчёт показывает количество изданий, экземпляров, активных и просроченных выдач, до 10 популярных книг и выдачи по факультетам за всё время.
F09	Журнал показывает 100 последних успешных изменений фонда, выдач и возвратов с исполнителем и временем UTC.
F10	GET /health проверяет доступ к базе и возвращает статус и версию. Команда backup создаёт согласованную резервную копию.

4.2 Правила данных

Книги содержат название, основного автора, год 1450–2100 и уникальный ISBN либо внутренний шифр. Экземпляр имеет уникальный инвентарный номер, книгу и филиал. Дата возврата и срок не могут предшествовать выдаче. Просрочка наступает на следующий день после due_at. Книги с экземплярами и экземпляры с историей не удаляются.

4 Требования к продукту продолжение

4.3 Требования к системе в целом

Архитектура: браузер, единое HTTP-приложение и реляционная SQLite. Требуются русскоязычный интерфейс, JSON API и сохранение данных между запусками. Интерфейс должен быть пригоден для настольного экрана и ширины 390 пикселей без горизонтального переполнения страницы; широкие таблицы допускают собственную прокрутку.

Область эксплуатации версии 0.1.0 — локальная учебная установка и небольшой демонстрационный фонд. Целевые объёмы для приёмки: не менее 4 книг, 8 экземпляров, 4 авторов, 2 филиалов и 3 факультетов. Промышленная отказоустойчивость, распределённая работа и высокая нагрузка в эту версию не входят.

4.4 Надёжность и целостность

Запись выдачи и возврата выполняется атомарно с аудитом. Внешние ключи и ограничения уникальности включаются в базу. Два одновременных запроса выдачи одного экземпляра должны приводить к одной успешной операции и одному отказу, без дубликатов. При ошибке транзакция откатывается.

Резервная копия создаётся средствами SQLite без перезаписи существующего файла. Восстановление производится после остановки приложения переключением DATABASE_PATH на сохранённую копию. Критерии проверки: integrity_check возвращает ok, каталог открывается и контрольные записи сохранены.

4.5 Безопасность и обработка ошибок

Пароль читателя имеет длину 10–128 символов; в БД хранится только хеш. Серверная сессия по умолчанию действует 8 часов; cookie использует HttpOnly и SameSite=Lax. Роль проверяется на сервере. Изменяющие запросы требуют CSRF-токен действующей сессии. Параметры SQL передаются отдельно от текста запросов.

Ошибки возвращаются в JSON с кодом и сообщением. Предусмотрены 400, 401, 403, 404, 405, 409, 413, 422 и 500. В ответе 500 отсутствуют детали внутреннего стека. Размер тела ограничен 64 КиБ. Чужой Origin отклоняется. Debug-режим выключен.

4.6 Виды обеспечения и программная среда

Информационное обеспечение: справочники и история в SQLite; схема описана в schema.sql. Лингвистическое: русский интерфейс, UTF-8. Программное: Python 3.11+, Flask 3.1.2, python-dotenv 1.1.1, HTML, CSS, JavaScript без сборщика. Техническое: компьютер с Python и браузером Chromium; интернет требуется для установки зависимостей. Конфигурация задаётся переменными окружения и файлом .env; образец поставляется без рабочих секретов.

Git и контейнерная поставка

4.7 Требования к Git и автоматическим проверкам

G01. Исходный код и документация хранятся в публичном репозитории <https://github.com/Draponoid/library-app>. Основная ветка — master. Для каждой функции оформляется задача с критериями готовности и отдельная рабочая ветка; не менее двух содержательных коммитов отражают ход работы.

G02. Изменение передаётся через pull request в master со ссылкой на задачу, описанием результата и проверок. До слияния выполняются ревью вторым участником и успешные автоматические проверки. Прямые изменения основной ветки обходят установленный порядок приёма.

G03. Необходимо воспроизвести конфликт слияния, сохранить исходные коммиты и объединить оба требования без маркеров конфликта. Разрешение фиксируется merge-коммитом и объясняется в docs/GIT_PROCESS.md.

G04. После приёма первой версии создаётся аннотированный тег v0.1.0 на проверенном коммите master и отправляется в удалённый репозиторий. VERSION должен содержать 0.1.0. Тег не переносится на другой коммит без отдельного согласованного решения.

G05. Файлы .env, пользовательские базы, резервные копии, .venv и настройки IDE исключаются через .gitignore. Проверяются и текущие отслеживаемые файлы, и история. В Git хранится только .env.example с образцами значений.

G06. GitHub Actions проверяет Python-тесты, синтаксис Python и JavaScript, тест интерфейсной логики, сборку и запуск Docker-контейнера, /health и главную страницу. Для успешного master публикуется проверенный образ в GHCR; публикация образа не означает развёртывание на сервере.

4.8 Контейнерная среда

Предусмотрены Dockerfile и compose.yml. В образе используется Python 3.13 и Waitress; приложение работает от непривилегированного пользователя. SQLite размещается в постоянном томе /data. Секреты передаются окружением и не копируются в образ. Приёмка контейнера подтверждается успешной стадией CI.

5 Состав и содержание работ

5.1 Порядок планирования. План отсчитывается от дня D0 — согласования ТЗ. Календарные даты утверждает преподаватель. Это план учебного цикла; фактический протокол проверки поставляемой версии приведён отдельно.

5.2 Этапы и результаты работ

Срок	Этап	Результат
D0–D1	Анализ и ТЗ	Роли, сценарии, правила, критерии приёмки.
D2–D3	Данные и API	Схема, начальные данные, аутентификация, справочники.
D4–D5	Выдачи и отчёты	Транзакции, ограничения, возврат, аудит и агрегаты.
D6–D7	Веб-интерфейс	Формы, поиск, обработка ошибок и адаптация.
D8	Испытания	Автоматические тесты и проверка в браузере.
D9–D10	Документация и сдача	Инструкция, ТЗ, отчёт, архив версии 0.1.0.

6 Порядок контроля и приёмки системы

6.1 Организация испытаний. Предварительные испытания выполняют разработчики на отдельной временной базе. В опытном использовании один участник действует как библиотекарь, второй — как читатель. Итоговую приёмку проводит преподаватель по поставляемому архиву и демонстрации.

6.2 Критерии приёмки

Объект контроля	Условие приёмки
F01–F02 и F07	Регистрация и вход работают; неверный пароль, истёкшая сессия и недопустимая роль дают отказ; чужие выдачи не раскрываются.
F03–F04	Операции пяти сущностей и комбинированный поиск воспроизводятся; связанные записи не удаляются.
F05–F06	Выдача/возврат сохраняют историю; повторные, просроченные и сверхлимитные операции отклоняются; конкурентная выдача даёт 201 и 409.
F08–F10	Отчёты соответствуют контрольным данным; аудит содержит действие; /health успешен; резервная копия целостна.
Комплект поставки	Успешны 22 Python-теста и тест JavaScript; выполнены проверки CI и контейнера; инструкция запуска воспроизводима; рабочие секреты исключены из поставки.

6.3 Устранение замечаний и передача результата. При дефекте, нарушающем целостность или права доступа, приёмка откладывается до исправления и повторной проверки. Дистрибутив передаётся архивом исходников и документации; итоговое решение и замечания фиксируются принимающей стороной.

6.4 Приёмка Git процесса

Проверяются G01–G06: задача связана с PR, история содержит рабочую ветку и содержательные коммиты, ревью и CI выполнены до слияния, конфликт воспроизводится по исходным коммитам, тег v0.1.0 доступен на GitHub, секреты и локальные файлы не отслеживаются. При отсутствии опубликованного тега Git-часть не считается полностью завершённой.

7 Требования к документированию

7.1 Форма и порядок передачи документов. Документы предоставляются на русском языке в электронном виде: отчёт с титульным листом и ТЗ в PDF, редактируемый текст в HTML и Markdown, исходные файлы приложения в UTF-8. Комплект передаётся единым ZIP-архивом. В интерфейсе, VERSION и документации используется номер 0.1.0.

7.2 Состав документации

Документ	Содержание
README.md	Назначение, запуск, конфигурация, роли, тесты, резервное копирование, ограничения.
docs/TECHNICAL_SPEC.md	Восемь разделов технического задания.
docs/API.md	Маршруты, права, входные поля, ответы, коды ошибок и пример запросов.
docs/DATABASE.md и schema.sql	Сущности, связи, ограничения и физическая схема.
docs/TEST_REPORT.md	Среда, перечень испытаний и фактические результаты.
docs/DEMO.md	Сценарий демонстрации и ответы на вопросы.
CONTRIBUTING.md	Правила изменения, обязательные команды, запреты и порядок приёмки.
docs/GIT_PROCESS.md	Задача, ветка, коммиты, PR, конфликт, ревью и выпуск тега.

8 Источники разработки

8.1 Учебные и методические материалы

1. Задание к лабораторной работе №1 «Сквозной проект и Git-процесс» и таблица тем: вариант 3 «Библиотека». Материалы предоставлены в виде изображений; учтено дополнение об аутентификации от 18.09.2026.
2. Методические материалы «Техническое задание к ЛР 1»: восемь разделов ТЗ и их наполнение. ГОСТ 34.602-89 указан в этих материалах как ориентир структуры; в данной работе применяется предоставленный учебный шаблон. IEEE Std 830 не выбран как основной шаблон.

8.2 Техническая документация

3. Документация Flask: <https://flask.palletsprojects.com/en/stable/> — устройство веб-приложения, маршруты и обработка запросов.
4. Документация Python, модуль sqlite3: <https://docs.python.org/3/library/sqlite3.html> — соединения, транзакции и резервное копирование.
5. SQLite, Partial Indexes: <https://www.sqlite.org/partialindex.html> — частичный уникальный индекс активной выдачи. Дата обращения к электронным источникам: 19.09.2026.
6. Репозиторий проекта: <https://github.com/Draponoid/library-app>; задача №1, PR №2, история и результаты GitHub Actions. Проверяемый снимок master: 9a510ba, 19.09.2026.

Проверка Git процесса

Снимок проверки: master, 9a510ba6077972ce326a560732e4f73b53bb4d3f. Проверка выполнена по публичному GitHub API, локальному клону и повторному запуску тестов.

Элемент	Подтверждение
Задача	Issue №1 «Фильтрация выдач по состоянию», закрыта PR №2.
Ветка и коммиты	lab/loan-status: 3d4fb0b, 9884bc1, 53c2ee7, 7607480. Ветка после слияния удалена; история сохранена в PR.
Слияние	PR №2 слит в master; merge-коммит 39b54a3.
Конфликт	Повторное merge-tree для 9884bc1 и 88d07d5 даёт конфликт docs/GIT_PROCESS.md; 53c2ee7 сохраняет оба требования.
CI	PR: запуск 35427587030 успешен. Текущий master: запуск 35433186360 успешен.
Ревью	Есть COMMENTED-ревью от Draconoid с описанием автоматизированной проверки и исправления. Независимое ревью второго студента не подтверждено.
Секреты	Проверенные локальные пути игнорируются и отсутствуют в просмотренной истории файлов.
Тег	На GitHub тега v0.1.0 нет; требуется создать и отправить после приёмки версии.

Ветка master не защищена правилами GitHub, поэтому технически возможно обойти PR и CI. Для учебной работы это рекомендация по усилению процесса, а не отдельный обязательный пункт исходного задания.

Ссылки: <https://github.com/Draconoid/library-app/issues/1> и <https://github.com/Draconoid/library-app/pull/2>. Актуальные исправления документов подготовлены локально; факт их публикации в репозитории этим отчётом не подтверждается.

Протокол испытаний

Проверяемый снимок GitHub: master, коммит 9a510ba от 19.09.2026. Повторно выполнены 22 Python-теста и один тест JavaScript. Проверки Python и JavaScript завершились успешно. GitHub Actions для этого коммита также успешен.

№	Проверка	Итог
1	Health, HTML и JavaScript	Пройден
2	Закрытый API без входа	Пройден
3	Вход, выход и хеш пароля	Пройден
4	Регистрация и запрет подмены роли	Пройден
5	Валидация регистрации	Пройден
6	Ограничения роли читателя	Пройден
7	CSRF и чужой Origin	Пройден
8	Истечение сессии	Пройден
9	CRUD пяти сущностей	Пройден
10	Неверные JSON, поля и типы	Пройден
11	Защита связанных данных от удаления	Пройден
12	Поиск и фильтры	Пройден
13	Выдача, возврат и история	Пройден
14	Лимит активных выдач	Пройден
15	Блокировка при просрочке	Пройден
16	Одновременная выдача одного экземпляра	Пройден
17	Изоляция выдач разных читателей	Пройден
18	Отчёты и аудит	Пройден
19	Целостность резервной копии	Пройден
20	Коды ошибок и защитные заголовки	Пройден
21	Фильтры состояния выдач и неверный статус	Пройден
22	Изоляция читателя при всех фильтрах	Пройден
JS	Защита от устаревшего ответа фильтра	Пройден

Базовый браузерный сценарий был проверен в первоначальной версии. В этой проверке повторно запускались автоматические тесты текущего master, включая фильтры выдач и защиту от устаревших ответов. Полный браузерный сценарий повторно не выполнялся.

Нагрузочное тестирование не проводилось. Docker проверен по успешной стадии GitHub Actions; локально контейнер не запускался. Результаты не заменяют приёмку преподавателем.

Запуск и демонстрация

Первый запуск в Windows

Распаковать архив, открыть терминал в папке library-app и выполнить команды по порядку:

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe setup_local.py
.venv\Scripts\python.exe app.py run
```

Открыть <http://127.0.0.1:5000>. Первичная настройка создаёт случайные пароли и выводит их в терминал; логины — librarian и reader. Пароли сохранены в локальном .env. Повторно выполняется только команда запуска сервера.

Демонстрация преподавателю

1. Показать /health, выполнить вход библиотекаря и поиск книги. 2. Добавить автора, книгу и экземпляр. 3. Оформить выдачу читателю, показать срок и недоступность экземпляра для новой выдачи. 4. Принять возврат, открыть историю, отчёты и журнал. 5. Войти как читатель и показать ограниченный доступ. 6. Показать задачу №1, ветку и коммиты PR №2, ревью, CI и конфликт. После публикации тега показать v0.1.0. 7. Запустить тесты и открыть ТЗ.

Резервное копирование

```
python app.py backup --output backups/library-copy.sqlite3
```

Для восстановления остановить сервер, сохранить текущую базу и переключить DATABASE_PATH на файл копии. После запуска проверить /health и каталог. Копию с пользователями и сессиями хранить отдельно от публичного комплекта.

Вывод

Получена работоспособная учебная версия библиотеки с веб-интерфейсом, реляционной моделью и HTTP API. Требования сопровождаются проверками, запуск воспроизводится по инструкции, правила выдачи защищены транзакциями и ограничениями базы. Подготовлены техническое задание и документация для дальнейшего развития проекта.

Git-процесс включён в работу. Перед сдачей необходимо опубликовать тег v0.1.0, подтвердить проверку вторым участником по правилам проекта и перенести актуальную документацию в репозиторий. ФИО преподавателя заполняется на титульном листе.